

Audion Disk Tools — User Guide

[Русский](#) · [About](#) · [Command Reference](#)

Contents

- [Running](#)
- [The Working Route](#)
- [The Safety Ladder](#)
- [Modes](#)
- [Safety Parameters](#)
- [Filters](#)
- [Cloud Storage](#)
- [Reports and Terminal](#)
- [Checks After Changes](#)
- [Technical Reference](#)

How to work with it: the route, the safety ladder, the modes, filters, cloud storage, reports.

Running

<code>launcher_project.cmd</code>	the main one
<code>launcher_profiles.cmd</code>	saved synchronisation profiles

Both come up through the quick picker and through a plain menu — if the first is missing, the second works.

The Working Route

At the top of the window are the source and the target. These are not two fixed project folders but any reachable paths: an external drive, a network share, a large directory.

button	what it does
Source	choose the source folder
Add file...	choose a single file
Target	choose the destination
Reset	restore the project folders, touching no files
Delete	clear the current source and target after confirmation
List	print the names of the current source files

The fields remember path history; a pinned path moves to the top.

The Safety Ladder

For folders that matter, four steps:

1. **Compare.** What actually differs. Nothing is written to disk.
2. **A dry run** of the chosen mode.
3. **Read the report:** what will be copied, updated, quarantined, deleted.
4. **The real run.**

For an everyday backup two usually suffice: a one-way update, and a mirror when a real mirror with deletion is what you want.

Modes

mode	direction	deletes	when
Folder check	one folder	no	integrity against a checksum manifest
Compare	source → target	no	before anything serious
One-way	source → target	no	ordinary transfer: new and changed
Mirror	source → target	yes	an exact copy, including removing the surplus
Mirror with quarantine	source → target	to quarantine	the same, but the surplus survives

mode	direction	deletes	when
Two-way	both ways	no	reconciling two working folders
Manifest and diff	source → target	per action	a filtered diff applied as update or mirror
Mask copy	source → target	no	a one-off selection, e.g. <code>*.docx</code> only

Safety Parameters

Comparison method

value	how it compares	when
<code>quick</code>	size and timestamp	fast, trusts metadata more
<code>safe</code>	plus a checksum on matching size with differing time	the default , for real work
<code>strict</code>	a checksum for every same-size file	when equal size and equal time matter

Operation policy

value	what happens to the surplus in the target
<code>copy_update</code>	it stays
<code>mirror_safe</code>	it moves to <code>_audion_quarantine</code>
<code>mirror_hard</code>	it is deleted after the earlier phases succeed

For a filtered hard mirror and a large share of deletions, the command line requires explicit guard flags.

The anchor

`anchor=true` in a profile requires an `.audion-anchor` file in both roots **before** the walk begins.

This is for roots that come over the network: a dropped Windows mount is presented as an existing, empty directory — and to a mirror that means "delete everything in the target".

The anchor is placed once, by a separate command:

```
runtime\python.exe system_core\main.py anchor N:\Projects
```

It never creates itself during a run. Otherwise the guard would fire for everything except the one case it exists for.

The pre-run check

Above the command fields the window shows a card: whether the anchor is in place, whether there is free space for the planned write, and how many files will be deleted — as a count and as a share of the target.

For mirror policies it also issues a confirmation bound to the plan you just read: what runs is exactly what was shown.

Filters

A filter is built from including and excluding rules:

rule	what it does
include masks	limit the working set
exclude masks	subtract from what was selected
presets	refer to <code>config\sync_presets.json</code>
excluded directories	remove <code>.git</code> , <code>node_modules</code> , caches
hidden	exclude dot-prefixed paths

In the window the manual extension picker is split into "Include" and "Exclude" tabs. **An empty selection means "leave the profile as it is", not "nothing"** — no runtime override is added.

Details: `tools\SYNC_PROFILES_RU.md` (Russian).

Cloud Storage

Ordinary paths go through the file engine. Real cloud storage is the data transfer section: folder, network share, server, and cloud are equals there, with one baseline and a list of machines, and the engine per pair chosen automatically.

The configuration is portable by default, inside the project. **Access tokens and the contents of that configuration are never written into project files.**

Installation, configuration transfer, diagnostics: `tools\RCLONE_PORTABILITY_RU.md` and `tools\RCLONE_OPERATIONS_RU.md`; transfer operations: `tools\TRANSFER_RU.md`. All Russian.

Reports and Terminal

Live output with Cyrillic and colour, a report per operation, the last report on one button.

A slow channel does not look like a freeze: progress is shown separately.

The workbench, terminal, tooltips, and caches: `tools\WORKBENCH_AND_GUI_RU.md`.

Checks After Changes

A short set of commands: the [checklist](#) (Russian).

Technical Reference

Where Things Live

<code>config\sync_presets.json</code>	filter presets
<code>config\sync_pairs.json</code>	saved folder pairs and policies
<code>config\rclone\rclone.conf</code>	portable cloud configuration

Deprecated

The `dry_run_default` key is ignored with a warning. Direction and deletion come from an explicit operation policy — which is why it was replaced: a dry run must not depend on a setting that is easy to forget.

Limits of the File Engine

Local and external drives, UNC network paths, mounted network folders, local folders of cloud clients, mounted remotes — everything Windows and Python see as an ordinary directory.

Workbench Naming

One shared vocabulary across all Audion projects: **Source**, **Add file...**, **Target**, **Reset**, **Delete**, **List**. In Russian: **Источник**, **Добавить файл...**, **Назначение**, **Сбросить**, **Удалить**, **Список**.

Reset restores the project folders and deletes no files. **Delete** clears the current source and target only after confirmation. The words **Destination**, **Clear**, «Цель», and «Очистить» are not used for these controls.